SOLUCIÓN CORREGIDA

Tarea 1 RA1

Persistencia en ficheros de texto

1. Introducción

Este documento recoge una solución corregida y organizada de la práctica “Tarea 1 RA1”, centrada en la persistencia en ficheros de texto usando Java.

La solución está pensada para alumnado de primer contacto con acceso a datos. Por eso se ha priorizado un enfoque simple: uso de la clase File para trabajar con rutas, directorios y ficheros; FileWriter y BufferedWriter para escribir; FileReader y BufferedReader para leer; y un borrado recursivo para eliminar directorios con contenido.

La práctica queda dividida en tres partes de código: el ejercicio 1, los ejercicios 2 y 3 juntos en un programa con menú y, por último, el ejercicio 4 dedicado al fichero Empleados.txt.

2. Estructura recomendada del proyecto

Paquete sugerido: com.ies.ad.ficherostexto

Archivo	Ruta
Ejercicio1CrearDirectorioYFichero.java	src/main/java/com/ies/ad/ficherostexto
Ejercicio2y3MenuFicheros.java	src/main/java/com/ies/ad/ficherostexto
Ejercicio4EmpleadosTexto.java	src/main/java/com/ies/ad/ficherostexto

3. Ejercicio 1 · Crear directorio y fichero

Objetivo: crear un directorio llamado miDirectorio dentro de C:\AD\Ejercicios y, dentro de él, un fichero llamado fichero_de_texto.txt.

Código completo

Ruta: src/main/java/com/ies/ad/ficherostexto/Ejercicio1CrearDirectorioYFichero.java

```
package com.ies.ad.ficherostexto;

import java.io.File;
import java.io.IOException;

public class Ejercicio1CrearDirectorioYFichero {

    public static void main(String[] args) {
        String rutaBase = "C:\\AD\\Ejercicios";
        File directorio = new File(rutaBase, "miDirectorio");
        File fichero = new File(directorio, "fichero_de_texto.txt");

        try {
            if (!directorio.exists()) {
                if (directorio.mkdirs()) {
                    System.out.println("Directorio creado: " + directorio.getAbsolutePath());
                } else {
                    System.out.println("No se pudo crear el directorio.");
                    return;
                }
            } else {
                System.out.println("El directorio ya existe: " + directorio.getAbsolutePath());
            }

            if (!fichero.exists()) {
                if (fichero.createNewFile()) {
                    System.out.println("Fichero creado: " + fichero.getAbsolutePath());
                } else {
                    System.out.println("No se pudo crear el fichero.");
                }
            } else {
                System.out.println("El fichero ya existe: " + fichero.getAbsolutePath());
            }

        } catch (IOException e) {
            System.out.println("Error de entrada/salida: " + e.getMessage());
        }
    }
}
```

Qué debe ocurrir al ejecutarlo

- Si la carpeta no existe, se crea.
- Si el fichero no existe, se crea vacío.
- Si ya existían, el programa lo informa por pantalla sin fallar.

4. Ejercicios 2 y 3 (juntos)

Aspectos importantes

- El menú debe repetirse hasta que el usuario pulse la opción de salir.
- Para borrar un directorio con contenido, delete() no es suficiente.
- Hace falta un método recursivo que recorra primero lo que hay dentro y lo borre antes de eliminar el directorio principal.
- Para escribir las provincias de Andalucía se puede usar BufferedWriter, que es cómodo y legible para escribir línea a línea.

Código completo

Ruta: src/main/java/com/ies/ad/ficherostexto/Ejercicio2y3MenuFicheros.java

```
package com.ies.ad.ficherostexto;

import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio2y3MenuFicheros {

    private static final String RUTA_BASE = "C:\\AD\\Ejercicios";

    private static final String[] PROVINCIAS_ANDALUCIA = {
            "Almería",
            "Cádiz",
            "Córdoba",
            "Granada",
            "Huelva",
            "Jaén",
            "Málaga",
            "Sevilla"
    };

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        File directorioBase = new File(RUTA_BASE);
```

```
        File miDirectorio = new File(directorioBase, "miDirectorio");
        File ficheroOriginal = new File(miDirectorio, "fichero_de_texto.txt");

        File nuevoDirectorio = new File(directorioBase, "nuevoDirectorio");
        File fichero2 = new File(nuevoDirectorio, "fichero_de_texto2.txt");

        if (!directorioBase.exists()) {
            directorioBase.mkdirs();
        }

        int opcion;

        do {
            mostrarMenu();
            System.out.print("Elige una opción: ");
            opcion = leerEntero(sc);

            switch (opcion) {
                case 1:
                    crearNuevoDirectorio(nuevoDirectorio);
                    break;
                case 2:
                    crearFicheroDentroDeNuevoDirectorio(nuevoDirectorio, fichero2);
                    break;
                case 3:
                    eliminarFicheroOriginal(ficheroOriginal);
                    break;
                case 4:
                    eliminarDirectorioRecursivo(nuevoDirectorio);
                    break;
                case 5:
                    escribirProvinciasEnFichero(fichero2, nuevoDirectorio);
                    break;
                case 6:
                    System.out.println("Saliendo del programa...");
                    break;
                default:
                    System.out.println("Opción no válida.");
            }

            System.out.println();

        } while (opcion != 6);

        sc.close();
    }

    private static void mostrarMenu() {
        System.out.println("===== MENÚ =====");
        System.out.println("1. Crear directorio llamado nuevoDirectorio");
        System.out.println("2. Crear fichero_de_texto2.txt en nuevoDirectorio");
        System.out.println("3. Eliminar fichero_de_texto.txt");
```

```
        System.out.println("4. Eliminar directorio nuevoDirectorio (aunque tenga contenido)");
        System.out.println("5. Escribir provincias de Andalucía en fichero_de_texto2.txt");
        System.out.println("6. Salir");
    }

    private static int leerEntero(Scanner sc) {
        while (!sc.hasNextInt()) {
            System.out.print("Introduce un número válido: ");
            sc.next();
        }
        return sc.nextInt();
    }

    private static void crearNuevoDirectorio(File nuevoDirectorio) {
        if (!nuevoDirectorio.exists()) {
            if (nuevoDirectorio.mkdirs()) {
                System.out.println("Directorio creado correctamente: "
                        + nuevoDirectorio.getAbsolutePath());
            } else {
                System.out.println("No se pudo crear el directorio.");
            }
        } else {
            System.out.println("El directorio ya existe.");
        }
    }

    private static void crearFicheroDentroDeNuevoDirectorio(File nuevoDirectorio, File fichero2)
{
        try {
            if (!nuevoDirectorio.exists()) {
                nuevoDirectorio.mkdirs();
            }

            if (!fichero2.exists()) {
                if (fichero2.createNewFile()) {
                    System.out.println("Fichero creado correctamente: "
                            + fichero2.getAbsolutePath());
                } else {
                    System.out.println("No se pudo crear el fichero.");
                }
            } else {
                System.out.println("El fichero ya existe.");
            }

        } catch (IOException e) {
            System.out.println("Error al crear el fichero: " + e.getMessage());
        }
    }

    private static void eliminarFicheroOriginal(File ficheroOriginal) {
        if (ficheroOriginal.exists()) {
            if (ficheroOriginal.delete()) {
```

```
            System.out.println("Fichero eliminado correctamente.");
        } else {
            System.out.println("No se pudo eliminar el fichero.");
        }
    } else {
        System.out.println("El fichero no existe.");
    }
}

private static void eliminarDirectorioRecursivo(File directorio) {
    if (!directorio.exists()) {
        System.out.println("El directorio no existe.");
        return;
    }

    if (borrarRecursivo(directorio)) {
        System.out.println("Directorio eliminado correctamente.");
    } else {
        System.out.println("No se pudo eliminar el directorio.");
    }
}

private static boolean borrarRecursivo(File ficheroODirectorio) {
    if (ficheroODirectorio.isDirectory()) {
        File[] archivos = ficheroODirectorio.listFiles();

        if (archivos != null) {
            for (File archivo : archivos) {
                if (!borrarRecursivo(archivo)) {
                    return false;
                }
            }
        }
    }
    return ficheroODirectorio.delete();
}

private static void escribirProvinciasEnFichero(File fichero2, File nuevoDirectorio) {
    try {
        if (!nuevoDirectorio.exists()) {
            nuevoDirectorio.mkdirs();
        }

        if (!fichero2.exists()) {
            fichero2.createNewFile();
        }

        try (BufferedWriter bw = new BufferedWriter(new FileWriter(fichero2))) {
            for (String provincia : PROVINCIAS_ANDALUCIA) {
                bw.write(provincia);
                bw.newLine();
            }
```

```
            }

            System.out.println("Provincias escritas correctamente en: "
                    + fichero2.getAbsolutePath());

        } catch (IOException e) {
            System.out.println("Error al escribir en el fichero: " + e.getMessage());
        }
    }
}
```

Resultado esperado

- La opción 1 crea el directorio nuevoDirectorio.
- La opción 2 crea fichero_de_texto2.txt dentro de ese directorio.
- La opción 3 elimina fichero_de_texto.txt si existe.
- La opción 4 elimina nuevoDirectorio incluso si tiene contenido.
- La opción 5 escribe las ocho provincias de Andalucía, una por línea.
- La opción 6 finaliza el programa.

5. Ejercicio 4 · Escribir y leer Empleados.txt

Objetivo: crear un fichero de texto llamado Empleados.txt, escribir un listado de 10 empleados numerados y leer después el contenido para mostrarlo por pantalla.

Idea principal

El enunciado pide hacerlo dos veces: primero con FileWriter y FileReader, y después con BufferedWriter y BufferedReader. Lo mejor es separar cada parte en métodos distintos para que el alumnado vea claramente la diferencia.

Código completo

Ruta: src/main/java/com/ies/ad/ficherostexto/Ejercicio4EmpleadosTexto.java

```
package com.ies.ad.ficherostexto;

import java.io.*;

public class Ejercicio4EmpleadosTexto {

    private static final String RUTA_BASE = "C:\\AD\\Ejercicios";
    private static final String NOMBRE_FICHERO = "Empleados.txt";

    public static void main(String[] args) {
        File directorioBase = new File(RUTA_BASE);
```

```
        if (!directorioBase.exists()) {
            directorioBase.mkdirs();
        }

        File fichero = new File(directorioBase, NOMBRE_FICHERO);

        System.out.println("=== PARTE 1: FileWriter / FileReader ===");
        escribirConFileWriter(fichero);
        leerConFileReader(fichero);

        System.out.println();

        System.out.println("=== PARTE 2: BufferedWriter / BufferedReader ===");
        escribirConBufferedWriter(fichero);
        leerConBufferedReader(fichero);
    }

    private static void escribirConFileWriter(File fichero) {
        try (FileWriter fw = new FileWriter(fichero)) {
            for (int i = 1; i <= 10; i++) {
                fw.write("Empleado " + i + System.lineSeparator());
            }
            System.out.println("Escritura con FileWriter completada.");
        } catch (IOException e) {
            System.out.println("Error al escribir con FileWriter: " + e.getMessage());
        }
    }

    private static void leerConFileReader(File fichero) {
        try (FileReader fr = new FileReader(fichero)) {
            int caracter;
            while ((caracter = fr.read()) != -1) {
                System.out.print((char) caracter);
            }
            System.out.println("Lectura con FileReader completada.");
        } catch (IOException e) {
            System.out.println("Error al leer con FileReader: " + e.getMessage());
        }
    }

    private static void escribirConBufferedWriter(File fichero) {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(fichero))) {
            for (int i = 1; i <= 10; i++) {
                bw.write("Empleado " + i);
                bw.newLine();
            }
            System.out.println("Escritura con BufferedWriter completada.");
        } catch (IOException e) {
            System.out.println("Error al escribir con BufferedWriter: " + e.getMessage());
        }
    }
```

```
    private static void leerConBufferedReader(File fichero) {
        try (BufferedReader br = new BufferedReader(new FileReader(fichero))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
            System.out.println("Lectura con BufferedReader completada.");
        } catch (IOException e) {
            System.out.println("Error al leer con BufferedReader: " + e.getMessage());
        }
    }
}
```

Diferencias FileWR y BufferedWR

- FileWriter y FileReader trabajan de forma directa con caracteres.
- BufferedWriter y BufferedReader añaden un búfer y resultan más cómodos para escribir y leer líneas completas.
- Con BufferedReader, el método readLine() hace más sencilla la lectura por líneas.

6. Problemas típicos y cómo corregirlos

Problema	Corrección
La ruta no existe	Crear primero C:\AD\Ejercicios con mkdirs() antes de trabajar con ficheros.
El fichero se quiere crear dentro de una carpeta inexistente	Crear antes el directorio contenedor y después el archivo.
delete() no borra un directorio con archivos dentro	Usar un método recursivo que borre primero el contenido y al final el directorio.
La ruta de Windows da error	Recordar que en Java las barras invertidas se escriben dobles: \\.
Se olvida cerrar el flujo	Usar try-with-resources para que Java lo cierre automáticamente.
Se mezclan lectura por caracteres y por líneas	Explicar que FileReader lee carácter a carácter y BufferedReader puede leer línea a línea.